

2026 제61회 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
※ 다음 사항을 유의하여 채점하십시오. 1) AWS의 지역은 ap-northeast-2를 사용합니다. 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다. 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다. 4) shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다. 5) 문제지와 채점지에 있는 < > 는 변수입니다. 해당 부분을 변경해 입력합니다. 6) 채점은 문항 순서대로 진행해야 합니다. 7) 삭제된 채점자료는 되돌릴 수 없음으로 유의하여 진행하며, 이의신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다. 8) 부분 점수가 있는 문항은 채점 항목에 부분 점수가 적혀져 있습니다. 9) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다. 10) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다. 11) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다. 12) 채점 시에는 별도로 제공한 채점 스크립트(mark.sh)를 실행하여 채점할 수 있습니다. 다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다. 채점 스크립트는 root 경로에 지정하도록 합니다. 13) 배포된 채점 스크립트(mark.sh) 는 ec2-user에 최상위 경로에 위치 하도록 합니다. 14) EKS 클러스터가 Private Endpoint Only 환경으로 구성되어 kubernetes API Server에 대한 접근이 VPC 내부로 제한되므로, kubectl 작업 및 채점은 VPC 내부의 worldpay-bastion 호스트에서 진행합니다. 15) 모든 채점 사항은 worldpay-bastion으로 ssh 접속 후 진행합니다.		

2. 채점기준표

1) 주요항목별 배점				직 종 명		클라우드컴퓨팅		
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제1과제	1	VPC			○		○	
	2	Bastion Server			○		○	
	3	Database			○		○	
	4	S3 Hosting			○		○	
	5	Container Registry			○		○	
	6	Load Balancer			○		○	
	7	Container Orchestration			○		○	
	8	CloudFront			○		○	
	9	Logging			○		○	
	10	Monitoring			○		○	
합 계			30					

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
1과제	1	VPC	1	VPC	1.0
			2	Subnet	1.0
			3	Routing table	1.0
	2	Bastion Server	2	Elastic IP	1.0
	3	Database	1	KMS Encryption	1.0
			2	DB configure	1.5
	4	S3 Hosting	1	KMS Encryption	1.0
			2	Static Files Upload	1.0
	5	Container Registry	1	ECR Configure	1.0
			2	Image Tag & Size	1.5
	6	Load Balancer	1	book-alb Configure	1.0
			2	grafana-alb Configure	1.0
	7	EKS	1	Cluster Configure	1.5
			2	Nodegroup Configure	1.0
			3	Pod Identity Configure	1.5
			4	API Test	1.5
	8	CloudFront	1	Distribution Configure	1.5
			2	Cloudfront Configure	1.5
	9	Logging	1	FluentBit Configure	1.0
			2	Log Group Configure	1.0
			3	Log Format Check	1.5
	10	Monitoring	1	Prometheus Configure	1.0
			2	Datasource Configure	1.0
			3	Dashboard Configure	1.5
			4	Realtime Metric & Log Check	1.5
	총점				

3) 채점내용

순번	사전준비	
0	1) worldpay-bastion 서버에 SSH 를 통해 접근합니다. ssh ec2-user@<worldpay-bastion-퍼블릭-IP> 패스워드: worldpay2026! 2) aws configure 를 입력하고 default.region 을 ap-northeast-2 로 설정합니다. aws configure set region ap-northeast-2 3) 채점 명령어에 사용될 환경 변수를 설정합니다. export BUCKET_NAME=worldpay-bucket-\$(aws sts get-caller-identity --query Account --output text)	
순번	채점항목	
1	1-1-A (명령어 입력)	aws ec2 describe-vpcs --filter Name=tag:Name,Values=worldpay-vpc --query "Vpcs[].CidrBlock"
	1-1-A (예상 출력) <u>정확히 일치</u>	["10.0.0.0/16"]
	1-2-A (명령어 입력)	aws ec2 describe-subnets --filter Name=tag:Name,Values=worldpay-public-subnet-a --query "Subnets[0].CidrBlock" ₩ ; aws ec2 describe-subnets --filter Name=tag:Name,Values=worldpay-public-subnet-c --query "Subnets[0].CidrBlock" ₩ ; aws ec2 describe-subnets --filter Name=tag:Name,Values=worldpay-isolated-subnet-a --query "Subnets[0].CidrBlock" ₩ ; aws ec2 describe-subnets --filter Name=tag:Name,Values=worldpay-isolated-subnet-c --query "Subnets[0].CidrBlock"
	1-2-A (예상 출력) <u>정확히 일치</u> <u>순서 중요</u>	"10.0.0.0/24" "10.0.1.0/24" "10.0.2.0/24" "10.0.3.0/24"
	1-3-A (명령어 입력)	for s in worldpay-isolated-subnet-c worldpay-isolated-subnet-a; do subnet=\$(aws ec2 describe-subnets --filters Name=tag:Name,Values=\$s --query 'Subnets[0].SubnetId' --output text) count=\$(aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=\$subnet --query 'length(RouteTables[0].Routes[?NatGatewayId!=null])' --output text) ["\$count" -gt 0] && echo "\$s: 1" echo "\$s: 0" done

	1-3-A (예상 출력) <u>정확히 일치</u> <u>순서 무관</u>	worldpay-isolated-subnet-c: 0 worldpay-isolated-subnet-a: 0
2	2-1-A (명령어 입력)	aws ec2 describe-instances --filter Name=tag:Name,Values=worldpay-bastion --query "Reservations[].Instances[].PublicIpAddress" aws ec2 describe-addresses --query "Addresses[].PublicIp"
	2-1-A (예상 출력) <u>순서 무관</u>	첫 번째 명령어에 출력된 IP가 두 번째 명령어 출력된 IP에 포함되었는지 확인합니다. ["54.116.172.195"] ["3.39.169.30", "54.116.172.195", "54.180.5.223"]
3	3-1-A (명령어 입력)	aws dynamodb describe-table --table-name Concerts --query "Table.SSEDescription.{Status:Status,SSEType:SSEType}" --output json && aws kms list-aliases --query "Aliases[?AliasName=='alias/worldpay-db-key'].AliasName" --output text
	3-1-A <u>정확히 일치</u>	{ "Status": "ENABLED", "SSEType": "KMS" } alias/worldpay-db-key
	3-2-A (명령어 입력)	TABLE_NAME=Concerts; aws dynamodb describe-continuous-backups --table-name \$TABLE_NAME --query 'ContinuousBackupsDescription.PointInTimeRecoveryDescription.PointInTimeRecoveryStatus' --output text; aws dynamodb describe-contributor-insights --table-name \$TABLE_NAME --query 'ContributorInsightsStatus' --output text; aws dynamodb describe-table --table-name \$TABLE_NAME --query 'Table.DeletionProtectionEnabled' --output text; aws dynamodb describe-table --table-name \$TABLE_NAME --query 'Table.BillingModeSummary.BillingMode' --output text
	3-2-A (예상 출력) <u>정확히 일치</u>	ENABLED ENABLED True PAY_PER_REQUEST
4	4-1-A (명령어 입력)	aws s3api get-bucket-encryption --bucket \$BUCKET_NAME --query "ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.S SEAlgorithm" --output text && aws kms list-aliases --query "Aliases[?AliasName=='alias/worldpay-s3-key'].AliasName [0]" --output text

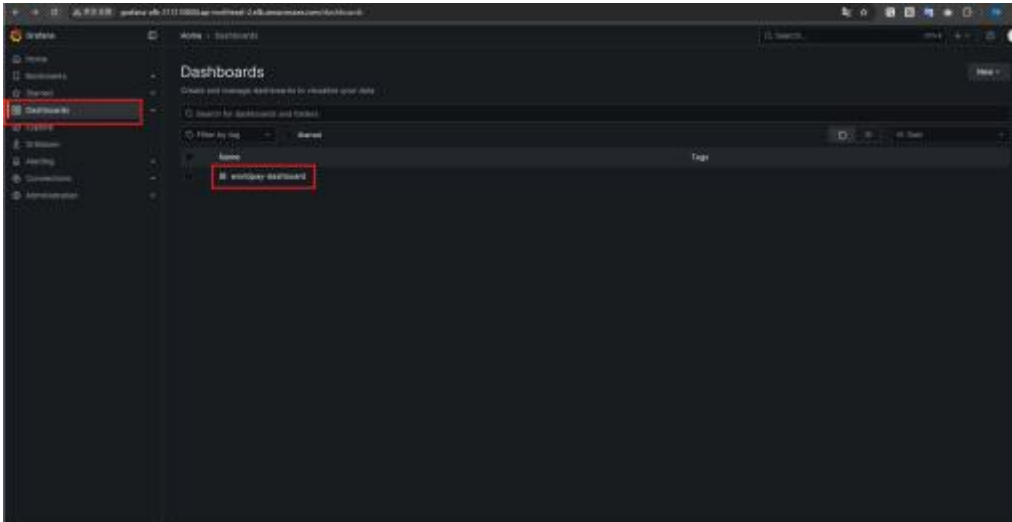
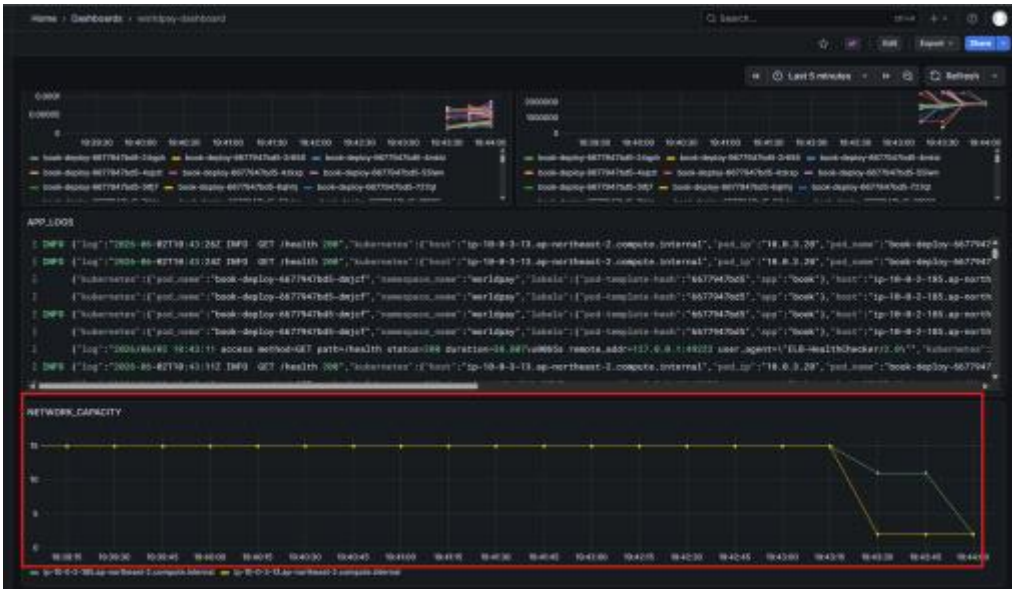
4	4-1-A <u>정확히 일치</u>	aws:kms alias/worldpay-s3-key
	4-2-A (명령어 입력)	aws s3 ls s3://\$BUCKET_NAME/ --recursive
	4-2-A (예상 출력)	index.html 과 main.jpeg 두 파일만 존재해야 합니다. 2026-05-11 01:13:46 1984 index.html 2026-05-11 01:13:47 180926 main.jpeg
5	5-1-A (명령어 입력)	aws ecr describe-repositories --repository-names worldpay-book --query "repositories[0].{RepositoryName:repositoryName,ScanOnPush:imageScanningConfiguration.scanOnPush}"
	5-1-A (예상 출력) <u>정확히 일치</u>	{ "RepositoryName": "worldpay-book", "ScanOnPush": true }
	5-2-A (명령어 입력)	SIZE=\$(aws ecr describe-images --repository-name worldpay-book --query 'imageDetails[0].imageSizeInBytes' --output text); TAG=\$(aws ecr describe-images --repository-name worldpay-book --query 'imageDetails[0].imageTags[0]' --output text); echo "\${W}ImageTagW":W"\$TAGW",W"SizeMBW":W"\${awk "BEGIN{printf W"%0.2fW", \$SIZE/1024/1024}" }W"}"
	5-2-A (예상 출력) <u>순서 무관</u>	태그가 v1.0.0 이어야 하고, 크기가 5MB) 이하 이어야 합니다. {"ImageTag": "v1.0.0", "SizeMB": "3.51"}
6	6-1-A (명령어 입력)	aws elbv2 describe-load-balancers --names book-alb --query "LoadBalancers[0].{LoadBalancerName:LoadBalancerName, Scheme:Scheme, Type:Type, State:State.Code}"
	6-1-A (예상 출력) <u>정확히 일치</u> <u>순서 중요</u>	{ "LoadBalancerName": "book-alb", "Scheme": "internal", "Type": "application", "State": "active" }
	6-2-A (명령어 입력)	aws elbv2 describe-load-balancers --names grafana-alb --query "LoadBalancers[0].{LoadBalancerName:LoadBalancerName, Scheme:Scheme, Type:Type, State:State.Code}"
	6-2-A <u>정확히 일치</u>	{ "LoadBalancerName": "grafana-alb", "Scheme": "internet-facing", "Type": "application", "State": "active" }

7	7-1-A (명령어 입력)	aws eks describe-cluster --name worldpay-cluster --query "cluster.{Version:version,EndpointPublicAccess:resourcesVpcConfig.endpointPublicA ccess,EndpointPrivateAccess:resourcesVpcConfig.endpointPrivateAccess}"
	7-1-A (예상 출력) <u>정확히 일치</u>	{ "Version": "1.35", "EndpointPublicAccess": false, "EndpointPrivateAccess": true }
	7-2-A (명령어 입력)	NG=\$(aws eks list-nodegroups --cluster-name worldpay-cluster --query "nodegroups[?starts_with(@, 'worldpay-nodegroup')] [0]" --output text); aws eks describe-nodegroup --cluster-name worldpay-cluster --nodegroup-name "\$NG" --query "nodegroup.{NodegroupName:'worldpay-nodegroup',InstanceTypes:instanceTypes, Status:status,DesiredSize:scalingConfig.desiredSize}"
	7-2-A (예상 출력)	DesiredSize가 2이상인지 확인합니다. { "NodegroupName": "worldpay-nodegroup", "InstanceTypes": ["t3.large"], "Status": "ACTIVE", "DesiredSize": 2 }
	7-3-B (명령어 입력)	kubect! get serviceaccount book-sa -n worldpay -o json jq '.metadata.name' aws eks list-pod-identity-associations --cluster-name worldpay-cluster W --query "associations[?serviceAccount=='book-sa' && namespace=='worldpay']"
	7-3-B (예상 출력)	빨간색 부분이 일치하는지 확인합니다. 파란색 부분은 달라도 무관합니다. "book-sa" [{ "clusterName": "worldpay-cluster", "namespace": "worldpay", "serviceAccount": "book-sa", "associationArn": "arn:aws:eks:ap-northeast-2:123412341234:podidentityassociation/worldpay-cluster /a-InmdfvxsZ0sjnagll", "associationId": "a-InmdfvxsZ0sjnagll" }]

	7-4-A (명령어 입력)	<pre>CF_DOMAIN=\$(aws cloudfront list-distributions --query "DistributionList.Items[0].DomainName" --output text); BOOKING_ID=\$(curl -s -X POST https://\$CF_DOMAIN/v1/book -H 'Content-Type: application/json' -d '{"client_id":"C001","username":"Alice","email":"alice@example.com","concert_name": "Seoul2026"}' jq -r '.booking_id'); echo "{\"BookingId\":\"\$BOOKING_ID\"}"; aws dynamodb get-item --table-name Concerts --key "{\"booking_id\":\"\$BOOKING_ID\"}" --query "Item.{BookingId:booking_id,S,ClientId:client_id,S,Username:username.S,Email:email. S,ConcertName:concert_name.S,CreatedAt:created_at.S}"</pre>
	7-4-A (예상 출력)	빨간색 부분이 일치하는지 확인합니다. 파란색 부분은 달라도 무관합니다. <pre>{"BookingId":"90R8KSSC"} { "BookingId": "90R8KSSC", "ClientId": "C001", "Username": "Alice", "Email": "alice@example.com", "ConcertName": "Seoul2026", "CreatedAt": "2026-05-31T06:53:26Z" }</pre>
8	8-1-A (명령어 입력)	<pre>DIST_ID=\$(for i in \$(aws cloudfront list-distributions --query 'DistributionList.Items[].Id' --output text);do ["\$(aws cloudfront list-tags-for-resource --resource arn:aws:cloudfront:\$aws sts get-caller-identity --query Account --output text):distribution/\$i --query 'Tags.Items[?Key=='Name'].Value[0]' --output text)" = "worldpay-cdn"]&&echo \$i&&break;done);aws cloudfront get-distribution --id "\$DIST_ID" --query "Distribution.{DefaultRootObject:DistributionConfig.DefaultRootObject,PriceClass:Di stributionConfig.PriceClass,Status:Status}"</pre>
	8-1-A (예상 출력) <u>정확히 일치</u> <u>순서 중요</u>	<pre>{ "DefaultRootObject": "index.html", "PriceClass": "PriceClass_All", "Status": "Deployed" }</pre>
	8-2-A (명령어 입력)	<pre>CF_DOMAIN=\$(aws cloudfront list-distributions --query "DistributionList.Items[0].DomainName" --output text); curl -s -o /dev/null -w "%{http_code}\n" https://\$CF_DOMAIN/; curl -s -X POST https://\$CF_DOMAIN/v1/book -H 'Content-Type: application/json' -d '{"client_id":"C002","username":"Bob","email":"bob@example.com","concert_name": "Busan2026"}' jq .</pre>
	8-2-A (예상 출력) <u>순서 중요</u>	파란색 부분은 달라도 무관합니다. <pre>200 { "booking_id": "XLFHEWTU" }</pre>

9	9-1-A (명령어 입력)	kubectl get daemonset worldpay-fluentbit -n logging -o json jq '{Name:.metadata.name,Namespace:.metadata.namespace}'
	9-1-A <u>정확히 일치</u>	{ "Name": "worldpay-fluentbit", "Namespace": "logging" }
	9-2-A (명령어 입력)	aws logs describe-log-groups --log-group-name-prefix /worldpay/application --query "logGroups[].{LogGroupName:logGroupName,RetentionInDays:retentionInDays}"
	9-2-A <u>정확히 일치</u>	[{ "LogGroupName": "/worldpay/application", "RetentionInDays": 7 }]
	9-3-A (명령어 입력)	CF_DOMAIN=\$(aws cloudfront list-distributions --query "DistributionList.Items[0].DomainName" --output text); curl -s -X POST https://\$CF_DOMAIN/v1/book -H 'Content-Type: application/json' -d '{"client_id":"C003","username":"Carol","email":"carol@example.com","concert_name":"Format2026"}' >/dev/null; sleep 30; aws logs filter-log-events --log-group-name /worldpay/application --filter-pattern POST --output json jq -r '.events sort_by(timestamp) last.message fromjson'
	9-3-A (예상 출력)	파란색 부분은 달라도 무관합니다. <pre>{ "log": "2026-06-02T08:26:39Z INFO POST /v1/book 200" "kubernetes": { "pod_name": "book-deploy-6677947bd5-dmjcf", "namespace_name": "worldpay", "labels": { "app": "book", "pod-template-hash": "6677947bd5" }, "host": "ip-10-0-2-185.ap-northeast-2.compute.internal", "pod_ip": "10.0.2.114" }, }</pre>

10	10-1-A (명령어 입력)	kubectl get pods -n monitoring -o wide grep prometheus awk '{print \$7}' xargs -l{} kubectl get node {} -o jsonpath='{.metadata.labels.topologyW.kubernetesW.io/zone}{"\n"}' sort -u
	10-1-A (예상 출력) <u>정확히 일치</u> <u>순서 무관</u>	ap-northeast-2a ap-northeast-2c
	10-2-A (명령어 입력)	GRAFANA_DNS=\$(aws elbv2 describe-load-balancers --names grafana-alb --query "LoadBalancers[0].DNSName" --output text); curl -s -u admin:worldpay2026! "http://\$GRAFANA_DNS/api/datasources" jq '[] select(.name=="worldpay-prometheus" or .name=="worldpay-logs") {Name:.name,Type:.type}'
	10-2-A <u>정확히 일치</u>	{ "Name": "worldpay-logs", "Type": "cloudwatch" } { "Name": "worldpay-prometheus", "Type": "prometheus" }
	10-3-A (명령어 입력)	GRAFANA_DNS=\$(aws elbv2 describe-load-balancers --names grafana-alb --query "LoadBalancers[0].DNSName" --output text); DASHBOARD_UID=\$(curl -s -u admin:worldpay2026! "http://\$GRAFANA_DNS/api/search" jq -r '[] select(.title=="worldpay-dashboard") .uid'); curl -s -u admin:worldpay2026! "http://\$GRAFANA_DNS/api/dashboards/uid/\$DASHBOARD_UID" jq '[.dashboard.panels[]].title'
	10-3-A <u>정확히 일치</u>	["POD_CPU", "POD_MEMORY", "APP_LOGS", "NETWORK_CAPACITY"]

10	10-4-A (콘솔 진행) <pre>GRAFANA_DNS=\$(aws elbv2 describe-load-balancers --names grafana-alb --query "LoadBalancers[0].DNSName" --output text)</pre> <pre>echo http://\$GRAFANA_DNS 브라우저를 통해 접속 합니다.</pre> <pre>echo username: admin</pre> <pre>echo password: worldpay2026!</pre>  <pre>kubectl scale deployment book-deploy -n worldpay --replicas=50</pre>
	10-4-A (예상 출력) 1분 이내에 대시보드에서 해당 요청에 대한 메트릭 변화가 인되어야 합니다.  NETWORK_CAPACITY 패널의 메트릭이 1분 이내에 감소하는 것을 확인합니다.